

1)

```
GNU nano 8.7.1 Virtual.c
#include <stdio.h>
int main(){
int x = 10;
printf("Value of x = %d\n",x);
printf("Virtual address of x = %p\n",(void *)&x);
return 0;
}
```

```
ketana_sindhu@KetaZone:~$ nano Virtual.c
ketana_sindhu@KetaZone:~$ gcc Virtual.c -o virtual
ketana_sindhu@KetaZone:~$ ./virtual
Value of x = 10
Virtual address of x = 0x7ffd670d1df4
ketana_sindhu@KetaZone:~$ |
```

2)

```
GNU nano 8.7.1 Variable.c
#include <stdio.h>
#include <stdlib.h>
int global = 10;
int bss_variable;
int main(){
int local = 20;
int *heap = malloc(sizeof(int));
*heap = 30;
printf("Code address : %p\n", (void *)main);
printf("Data address : %p\n", (void *)&global);
printf("BSS address : %p\n", (void *)&bss_variable);
printf("Heap address : %p\n", (void *)heap);
printf("Stack address : %p\n", (void *)&local);

free(heap);

return 0;
}
```

```
ketana_sindhu@KetaZone:~$ nano Variable.c
ketana_sindhu@KetaZone:~$ gcc Variable.c -o variable
ketana_sindhu@KetaZone:~$ ./variable
Code address : 0x64c2646991a9
Data address : 0x64c26469c010
BSS address : 0x64c26469c018
Heap address : 0x64c273331010
Stack address : 0x7ffffeb3d634c
ketana_sindhu@KetaZone:~$ |
```

3)

```
GNU nano 8.7.1 DemandPage.c
#include <stdio.h>
int main(){
int page_table[4] = {1, -1, 3, -1};
int page;
printf("Enter page number (0-3): ");
scanf("%d", &page);
if(page_table[page] == -1){
printf("Page Fault!\n");
printf("Page must be loaded into memory.\n");
}
else{
printf("Page is already in memory.\n");
printf("Physical frame = %d\n",page_table[page]);
}
return 0;
}
```

```
ketana_sindhu@KetaZone:~$ nano DemandPage.c
ketana_sindhu@KetaZone:~$ gcc DemandPage.c -o demand
ketana_sindhu@KetaZone:~$ ./demand
Enter page number (0-3): 1
Page Fault!
Page must be loaded into memory.
ketana_sindhu@KetaZone:~$ |
```

```
ketana_sindhu@KetaZone:~$ ./demand
Enter page number (0-3): 4
Page is already in memory.
Physical frame = 0
ketana_sindhu@KetaZone:~$ |
```

4)

```
GNU nano 8.7.1 AddressTranslation.c
#include <stdio.h>
int main(){
int page_table[] = {3, 5, 1, 7};
int page_size = 100;

int virtual_address = 250;

int page = virtual_address / page_size;
int offset = virtual_address % page_size;

int frame = page_table[page];

int physical_address = frame * page_size + offset;

printf("Virtual Address = %d\n",virtual_address);
printf("Page Number = %d\n",page);
printf("Offset = %d\n",offset);
printf("Frame Number = %d\n", frame);
printf("Physical Address = %d\n",physical_address);

return 0;
}
```

```
ketana_sindhu@KetaZone:~$ nano AddressTranslation.c
ketana_sindhu@KetaZone:~$ gcc AddressTranslation.c -o ad
ketana_sindhu@KetaZone:~$ ./ad
Virtual Address = 250
Page Number = 2
Offset = 50
Frame Number = 1
Physical Address = 150
ketana_sindhu@KetaZone:~$ |
```

5)

```
GNU nano 8.7.1 MemoryAllocate.c
#include <stdio.h>
#include <stdlib.h>

int main(){
int *p;
p = malloc(5 * sizeof(int));
if(p==NULL){
printf("Memory allocation failed\n");
return 1;
}
for(int i = 0;i<5;i++){
printf("%d ",p[i]);
}
printf("\n");
free(p);
return 0;
}
```

```
ketana_sindhu@KetaZone:~$ nano MemoryAllocate.c
ketana_sindhu@KetaZone:~$ gcc MemoryAllocate.c -o malloc
ketana_sindhu@KetaZone:~$ ./malloc
0 0 0 0 0
ketana_sindhu@KetaZone:~$ |
```

6)

```
GNU nano 8.7.1 Segmentation.c
#include <stdio.h>
int main(){
int *p = NULL;
printf("Before accessing memory\n");
*p = 100;
printf("After accessing memory\n");
return 0;
}
```

```
ketana_sindhu@KetaZone:~$ nano Segmentation.c
ketana_sindhu@KetaZone:~$ gcc Segmentation.c -o seg
ketana_sindhu@KetaZone:~$ ./seg
Before accessing memory
Segmentation fault ./seg
ketana_sindhu@KetaZone:~$ |
```